



STARVNT CORE

AUTOMATION SYSTEM FOUNDATION V1

Event-driven foundation for rules, actions, alerts and future Aura+ orchestration

MVP-first | Modular | Production-oriented | Future-ready

StarVnt Entertainment

Document Purpose

This document defines the V1 foundation for StarVnt Core Automation. The objective is not to build a large automation platform immediately. The objective is to create a clean event-driven layer that can detect business events, evaluate rules and conditions, execute controlled actions, record the outcome, and remain extensible for future Aura+ intelligence and orchestration.

North Star: Business Event -> Automation Trigger -> Condition Evaluation -> Rule -> Action -> Activity / Log

The architecture should build on the Customer Module and Customer Tracking foundation already established in StarVnt Core. Automation is a separate capability layer; it should react to business events rather than embed automation logic directly inside customer controllers or models.

Document outcome

- Provide a practical Day 1 / foundation blueprint for automation.
- Define event, rule, condition, action, job and log concepts.
- Demonstrate a first working automation use case around overdue follow-ups.
- Preserve clean boundaries for future Aura+ intelligence and recommendations.
- Keep the system modular enough to evolve without rewrites.

1. Core Automation Concept

StarVnt Core should progressively evolve from a passive record system into an operating layer that can react to meaningful customer and commerce events. The initial automation architecture is intentionally small, deterministic and auditable.

Business Event -> Automation Trigger -> Condition Evaluation -> Automation Rule -> Action Execution -> Activity / Log

Example: Customer Status Changed -> Automation Engine detects event -> rule checks whether the status is Quotation Sent -> condition matches -> create follow-up action -> activity recorded.

2. Long-Term Automation Vision

The first version establishes patterns for several future workflows:

Follow-up Automation - Follow-up date arrives -> system checks status -> if still pending, create alert / task.

Overdue Follow-up Detection - Follow-up date passes -> pending record is detected -> mark overdue/missed -> create activity -> generate alert -> optionally expose the event to Aura+ later.

Customer Inactivity Detection - No qualifying activity for X days -> customer flagged inactive -> alert generated -> future recommendation can be supplied by Aura+.

Status-Based Automation - Customer becomes Quotation Sent -> automatically create a follow-up reminder after a configurable delay.

Conversion Automation - Customer becomes Converted -> create activity -> future onboarding workflow -> Aura+ can analyze conversion patterns.

3. Automation Module Architecture

Automation must remain separate from Customer business logic. Exact folders can differ, but the responsibilities should remain distinct.

Suggested structure

automation/ triggers/ rules/ conditions/ actions/ services/ jobs/ events/ logs/ modules/

Automation Engine responsibility: coordinate Event -> Matching Rules -> Condition Evaluation -> Action Execution.

Business services should remain responsible for business operations. Controllers should remain responsible for request/response handling. Automation services should orchestrate reactions to already-completed business operations.

4. Event System Foundation

Initial business events should be standardized so other modules can react without hard-coded cross-module dependencies.

Event	Meaning	Typical Source
CUSTOMER_CREATED	New customer record created	Customer module
CUSTOMER_STATUS_CHANGE D	Customer status changed	Customer service
NOTE_CREATED	A customer note was added	Customer notes module
FOLLOW_UP_CREATED	A follow-up was scheduled	Follow-up service
FOLLOW_UP_COMPLETED	A follow-up was completed	Follow-up service
FOLLOW_UP_OVERDUE	A pending follow-up crossed its due time	Scheduler / follow-up job
CUSTOMER_INACTIVE	No qualifying activity for configured period	Scheduled job
QUOTATION_SENT	Quotation sent to customer	Future commerce module
CUSTOMER_CONVERTED	Customer converted	Future commerce / CRM integration
CUSTOMER_LOST	Customer marked lost	CRM / customer workflow

Business modules emit events. The automation engine consumes them and remains decoupled from the originating controller or database write.

5. Automation Rule Structure

Rules should begin simple and can later become database-driven. The important part is separating the trigger, conditions and actions so the design can evolve.

Logical rule model

Field	Purpose
ruleId	Stable identifier for the automation rule
name	Human-readable rule name
triggerEvent	Event that starts evaluation
conditions	One or more conditions to evaluate
actions	One or more actions to execute
status	Active / Inactive
priority	Ordering when several rules match
createdAt / updatedAt	Lifecycle metadata

Example Rule

Name: Follow-up Reminder
 Trigger: FOLLOW_UP_CREATED
 Condition: Follow-up date is within 24 hours
 Action: CREATE_ALERT / CREATE_ACTIVITY
 Status: Active

For V1, a small set of code-backed rules is acceptable. Do not introduce a full visual workflow builder until actual usage justifies it.

6. Condition Engine

A trigger should not execute every action automatically. Conditions create the decision boundary.

Example A: CUSTOMER_STATUS_CHANGED -> New Status == Quotation Sent -> TRUE -> Execute reminder action.

Example B: FOLLOW_UP_CREATED -> Follow-up Date < Current Date -> TRUE -> Create overdue activity.

Condition evaluation should be isolated from controllers, frontend code and persistence models. This gives the system a reusable decision layer for future rules.

Suggested condition categories

- Equality / inequality
- Date comparisons
- Status comparisons
- Boolean flags
- Thresholds / numeric comparisons
- Presence / absence checks

7. Automation Actions

Actions are the operational output of an automation decision. V1 should keep actions small, deterministic and auditable.

Action Type	Example
CREATE_ACTIVITY	Automation triggered after Customer status changed
CREATE_ALERT	Follow-up overdue / attention required
UPDATE_STATUS	Pending -> Missed / Overdue
CREATE_FOLLOW_UP	Create reminder after quotation
Future: SEND_EMAIL	Provider-independent notification action
Future: SEND_WHATSAPP	Provider-independent communication action
Future: SEND_NOTIFICATION	In-app / push notification
Future: ASSIGN_TEAM_MEMBER	Route work to a responsible user
Future: CREATE_TASK	Create an internal task
Future: TRIGGER_WEBHOOK	Notify another approved system
Future: TRIGGER_AURA_ANALYSIS	Request intelligence/recommendation from Aura+

8. Automation Execution Flow

Customer Action -> Business Service -> Event Generated -> Automation Engine -> Find Matching Rules -> Evaluate Conditions -> Execute Actions -> Create Activity / Log -> Return Result

Controllers should not become large automation coordinators. The business service emits the business event; automation receives the event and performs its own orchestration.

9. First Working Use Case: Follow-up Automation

The first practical automation flow should focus on overdue follow-ups because it uses data already present in the Customer Tracking layer and demonstrates event-driven automation plus scheduled detection.

Flow A - Follow-up created

Customer -> Follow-up Scheduled -> FOLLOW_UP_CREATED -> Automation Engine -> Evaluate Rule -> CREATE_ACTIVITY

Activity: "Follow-up scheduled"

Flow B - Follow-up becomes overdue

Scheduled Job -> Finds Pending Follow-up -> Due Date Passed -> Update Status -> Create Activity -> Create Alert -> Aura+ Ready

Expected operational outcome

Record	Expected Result
Follow-up	Status becomes Missed / Overdue
Activity	Automatic activity explaining the status transition
Alert	Customer follow-up requires attention
Automation Log	Execution, rule, customer and action trace recorded

10. Scheduled Job Foundation

Some events arise directly from user actions; others arise because time has passed. Therefore the automation layer needs a small job/scheduler abstraction.

Suggested flow

Scheduler -> Scheduled Job -> Fetch Relevant Records -> Evaluate Conditions -> Trigger Automation

Example periodic job: check pending follow-ups, compare due date to current time, transition eligible records, emit FOLLOW_UP_OVERDUE, and allow matching rules to generate an alert.

Keep scheduling simple in V1. A single worker / cron-style mechanism is enough unless actual scale proves otherwise. The architecture should keep the job interface replaceable.

11. Automation Logging

Every automation execution should eventually be traceable. This becomes essential once multiple rules and actions exist.

Field	Purpose
logId	Execution identifier
ruleId	Rule that was evaluated/executed
triggerEvent	Event that caused evaluation
customerId	Related customer when applicable
action	Action attempted/executed
status	Success / Failed / Skipped
timestamp	Execution time
error	Failure detail when relevant
correlationId	Trace across requests/events/jobs

Example

Rule: Overdue Follow-up Detection
Customer: Rahul Sharma
Action: Mark Follow-up as Missed
Status: Success
Timestamp: 2026-09-02 10:00 AM

Automation logs are for execution traceability. They are not the same as customer-facing activity history, even though an automation may create a corresponding customer activity entry.

12. Frontend Automation Readiness

No separate Automation Dashboard is required for the foundation stage.

However, the frontend architecture should allow future surfaces for:

- Automation Alerts
- Pending Actions
- Overdue Follow-ups
- Automation History

For now, Customer Details can simply surface operational outcomes such as: Follow-up overdue and Follow-up automatically marked as missed.

The UI should consume automation results through normal APIs; it should not contain independent business rules that duplicate the backend.

13. Aura+ Integration Boundary

Automation and Aura+ should remain separate capabilities with different responsibilities.

Layer	Primary Responsibility
StarVnt Core Automation	Events, deterministic rules, conditions, operational actions
Aura+	Intelligence, analysis, scoring, recommendations, predictions
Core Commerce	Authoritative business records and transactions

Conceptual future loop: Business Data -> Aura+ Analysis -> Recommendation -> Automation Engine -> Action

Example: Aura+ detects that a high-value customer is becoming inactive -> recommendation: contact within 24 hours -> future automation creates a follow-up task. The separation is intentional: Aura+ thinks. Automation executes. StarVnt Core operates.

14. Data Model / Entity Readiness

Automation should reference existing customer entities instead of duplicating them. Suggested foundation entities:

Entity	Purpose
AutomationRule	Trigger, conditions, actions, status, priority
AutomationExecution / AutomationLog	Execution trace
AutomationEvent	Normalized event envelope or event record
ScheduledJob	Job configuration / execution metadata
Alert	Operational signal generated by automation

Where Customer / Note / Follow-up / Activity entities already exist, automation should use the authoritative relationships to those records.

15. Security & Reliability Principles

Automation executes actions on customer data. Security and reliability therefore belong in the foundation, even though V1 remains intentionally small.

Minimum controls

- Automation events should be treated as validated internal inputs, not trusted raw browser data.
- Critical business actions must pass normal server-side authorization and business validation.
- Automation rules should not provide a hidden path around existing permissions.
- Every important execution should be traceable through logs / correlation IDs.
- Duplicate event delivery should not produce duplicate actions; action handlers should be idempotent where practical.
- A failed automation should not corrupt the underlying Customer record or block a primary customer transaction.
- Scheduled jobs should be retry-safe and observable.
- Sensitive customer information should not be copied into automation metadata unnecessarily.

16. Idempotency & Duplicate Protection

Automation is likely to encounter retries, delayed jobs or duplicate event deliveries. The architecture should therefore support a stable event identifier and execution key.

Example

```
Event: FOLLOW_UP_OVERDUE
Event ID: EVT-12345
Rule: OVERDUE_FOLLOWUP_V1
Execution Key: EVT-12345:OVERDUE_FOLLOWUP_V1
```

If the same event is received again, the engine should recognize that the matching rule has already executed successfully and avoid creating a second alert or duplicate status transition.

17. Failure Handling

The automation layer is downstream intelligence/operations infrastructure. Its failure should not break the core customer operation that produced the underlying event.

Failure	Expected Behaviour
Automation service temporarily unavailable	Primary Customer action still succeeds; event can be retried/reprocessed
Rule evaluation fails	Record execution failure and keep source business record intact
Action fails	Retry when safe; log final failure
Duplicate event arrives	Deduplicate / idempotently ignore if already processed
Scheduler fails temporarily	Next run should recover missed eligible records safely
Aura+ unavailable in future	Deterministic Core automation remains functional

The goal is graceful degradation rather than perfect distributed infrastructure on Day 1.

18. Implementation Sequence

Recommended build order for the first automation layer:

1. Create automation module boundaries and service interfaces.
2. Define event envelope and initial business event constants.
3. Implement Automation Engine service.
4. Implement simple rule matching and condition evaluation.
5. Implement CREATE_ACTIVITY and CREATE_ALERT actions.
6. Add Follow-up overdue scheduled job.
7. Add Automation Log / execution trace.
8. Wire Customer Follow-up events into the automation engine.
9. Demonstrate end-to-end overdue follow-up flow.
10. Document next-step extensions for database-driven rules, notifications and Aura+.

19. Minimum Automation Working Proof

At least one automation flow should be demonstrable end-to-end.

Recommended flow

Create Customer -> Create Follow-up -> Follow-up Due Date Passes -> Automation Job Detects Pending Follow-up -> Status Updated -> Activity Automatically Created -> Alert Generated

Expected result

Check	Expected
Automation module exists	YES
Business event emitted	YES
Rule matched	YES
Condition evaluated	YES
Action executed	YES
Activity created	YES
Alert generated	YES
Execution logged	YES
No duplicate action on retry	YES / validated
Aura+ integration boundary preserved	YES

This is the minimum evidence that the foundation is real rather than documentation-only.

20. End-of-Day Automation Report

The implementation owner should return a concise execution report using the following structure:

Completed

- ...

Working Automation

- ...

Events Implemented

- ...

Rules Implemented

- ...

Actions Implemented

- ...

Scheduled Jobs

- ...

Working Proof

- Event -> Rule -> Condition -> Action -> Activity / Alert

GitHub

Repository: ...

Branch: ...

Latest Commit: ...

Pending / Blockers

- ...

Next Recommendation

- ...

21. Success Criteria

The Automation Foundation should be considered successful when the following are true:

- Automation module architecture exists and is separate from Customer controllers.
- Business events can be generated from completed domain operations.
- Automation Service / Engine exists.
- Rules are separated from controllers and frontend code.
- Conditions can be evaluated independently.
- Actions can be executed and audited.
- At least one automation flow works end-to-end.
- Activity is generated automatically by the automation layer.
- Scheduled jobs are architected and at least one job is usable.
- Execution logs are available for troubleshooting.
- Future Aura+ integration is possible without coupling the automation engine to AI logic.
- Code is pushed to GitHub with a clear branch/commit.

22. StarVnt Platform Evolution

The intended progression is:

Customer Data -> Customer Tracking -> Activities + Follow-ups -> Automation -> Aura+ Intelligence -> Recommendations -> Automated Actions -> StarVnt Business Operating System

Architectural principle: Build one reliable layer at a time. Core records are authoritative. Automation reacts. Aura+ understands. The combined system becomes an operating brain without turning every module into a tightly coupled automation engine.

23. Final Engineering Directive

Do not build everything at once. Keep the first automation version small, testable and auditable. Reuse the Customer Tracking foundation. Introduce events and execution boundaries before introducing complexity. The architecture should make later notification, workflow, Revenue CRM and Aura+ integrations possible without rewriting the core.

Final North Star

Detect -> Decide -> Act -> Record -> Learn

Core Automation handles deterministic operations. Aura+ adds intelligence. Core Commerce remains the source of truth for business transactions. Every layer should expose only the data and actions required by its responsibility.

Prepared for StarVnt Engineering | StarVnt Entertainment